

Assignment 1: Recommending Code Tokens via N-gram Models

Lily Walker

Prof. Antonio Mastropaolo

github.com/lgwalker/GenAI_N-grams

February 20, 2026

Abstract

This report investigates the effectiveness of N-gram probabilistic language models for recommending code tokens in Java methods. By mining software repositories with high star counts and modern push dates, a diverse corpus was produced and preprocessed to evaluate models across various training set sizes (15k, 25k, and 35k tokens) and context windows ($n \in \{3, 5, 7\}$). Contrary to initial hypotheses, experimental results demonstrate that smaller training sets and shorter context windows (3-gram) yield superior performance, achieving a minimum perplexity of 40.79. Larger models suffered from significant data sparsity and vocabulary explosion. The final 3-gram model proved robust during inference, maintaining stable performance across both benchmark and self-mined test sets.

1 Mining Software Repositories

Repositories were mined using the pipeline format provided in Minilab 1. The complete preprocessing pipeline is documented in [PreprocessingData.ipynb](#). While the initial setup remained consistent with the lab, I adjusted the star count threshold to 500. This threshold was chosen to ensure high-quality code and to manage the volume of scraped repositories. Forks were excluded to prevent code duplication.

Additionally, I implemented a filter to fetch only repositories pushed after 2020 to ensure the inclusion of modern Java standards. My reasoning was that repositories inactive since 2014 or so likely contain legacy Java syntax that has been superseded by more recent versions. While the cloning process mirrored the minilab, I added further filtering for methods. In addition to removing non-ASCII characters, duplicates, and methods with fewer than 10 tokens, I introduced a maximum token count. This was based on the premise that excessively long methods often indicate lower modularity and quality. Finally, a unique token filter was applied to reduce noise from repetitive identifiers.

1.1 Splitting for Training and Validation

After extraction, the dataset was divided into three training sets (T1, T2, and T3) containing 15k, 25k, and 35k tokens respectively, structured such that $T1 \subset T2 \subset T3$. This setup was designed to test the hypothesis that increasing training data size consistently lowers perplexity. Prior to splitting, the data was shuffled to ensure the N-gram model generalizes well across diverse coding styles. Since GitHub data is naturally clustered by repository, shuffling was essential to prevent repository bias. I ensured each split contained a representative mix of projects rather than being skewed toward a single coding style. For evaluation, the test and validation sets were sliced from the head of the list to ensure the model was tested on truly unseen data.

2 Training the N-gram Model

The implementation leverages the Natural Language Toolkit (NLTK) library, specifically the `nltk.lm` module, to manage probabilistic language modeling. NLTK was selected for its robust support of smoothing techniques and its infrastructure for handling large-scale tokenized corpora. A key component is the `Vocabulary` class, which automates token frequency tracking and manages the transition of rare tokens into the `<UNK>` category. This ensures that probability mass is correctly distributed across known and unknown tokens.

The training pipeline utilizes NLTK’s `padded_everygram_pipeline`, which transforms token lists into a lazy-loading generator. This allows the model to learn both n -th order relationships and the lower-order backoff distributions required for smoothing. Using the `Lidstone` class allowed for precise control over the smoothing parameter (α), enabling the assignment of small probabilities to unseen sequences. Standardized NLTK methods for perplexity and log-score were used to ensure consistent evaluation. The full implementation is available in [Training.ipynb](#).

2.1 Evaluation and Analysis

The models were evaluated using perplexity across three training sizes and three context windows ($n \in \{3, 5, 7\}$). Contrary to the hypothesis that larger windows would yield better results, the 3-gram model trained on T1 (15k) performed best with a perplexity of 40.79. The sharp increase in perplexity for $n = 5$ and $n = 7$ (reaching 1231.92) indicates a significant data sparsity issue. These findings suggest that 5-token and 7-token sequences are often too unique to recur across different methods; consequently, the model relies heavily on smoothing, which dilutes probability mass. Furthermore, the decline in performance from T1 to T3 suggests a "vocabulary explosion," where added data introduced enough unique, low-frequency identifiers to make the distribution more uncertain to the model. The 3-gram T1 model was selected for its balance of capturing Java syntax without being overwhelmed by noise.

3 Evaluation and Inference Pipeline

The final stage involved implementing an [inference script](#) to evaluate the selected model on unseen data. Built with the `argparse` library, this script allows for flexible testing across datasets. It loads the best model and vocabulary artifacts and replicates the preprocessing environment by mapping tokens to the learned vocabulary and handling out-of-vocabulary (OOV) terms. To provide a granular view of performance, the engine records both ground-truth and predicted tokens for every position. This data, alongside associated probabilities and global perplexity, is exported in a standardized JSON format.

3.1 Testing Results and Comparative Analysis

The selected 3-gram model was evaluated on the provided benchmark set and a self-mined collection of approximately 1,000 Java methods. The results are summarized in Table 1.

The lower perplexity on the provided test set (36.00) indicates that the model effectively captures common Java syntax patterns. Its superior performance on this benchmark compared to the validation set suggests that the benchmark contains a high concentration of standard Java structures that align well with the model’s learned distribution.

The slightly higher perplexity on the self-mined set (41.34) is likely due to distribution shift; new repositories introduce project-specific naming conventions and diverse library calls. However, the small difference (5.34 points) demonstrates the robustness of the pipeline. This consistency indicates the model has learned general Java structural patterns rather than overfitting to specific training repositories.

Table 1: N-gram Model Perplexity Results

Training Set	Tokens	3-gram	5-gram	7-gram
T1 (Selected)	15,000	40.7931	223.7790	827.8859
T2	25,000	48.6296	282.6958	1041.9770
T3	35,000	54.9678	333.7070	1231.9175
Final Test Evaluation (Best Model)		Perplexity Score		
Provided Test Set (<code>results-xxxxxx.json</code>)		36.00		
Self-Mined Test Set (<code>results-yyyyyy.json</code>)		41.34		

4 Conclusion

The findings in this project reveal that while N-gram models are effective at capturing local Java syntax, they are highly sensitive to data sparsity and vocabulary scale. The superior performance of the 3-gram model on the smallest dataset (T1) contradicts the initial "more is better" hypothesis. This highlights that for traditional probabilistic models, increasing the context window or training size without adequate abstraction

leads to a "vocabulary explosion." This flattens the probability distribution and increases perplexity.

If this study were repeated, I would implement some changes to the data pipeline. First, I would use identifier abstraction to replace user-defined variable and method names with generic placeholders (eg. `VAR.1`). This would allow the model to focus on the structure of the language rather than project-specific identifiers, potentially making the 5-gram or 7-gram models more stable.

Code Link: The complete repository including notebooks and scripts can be found at:
https://github.com/lgwalker/GenAI_N-grams